

## **Experiment No: 6 - Establish local blockchain using Truffle**

### **AIM:**

Introduction to Truffle, establish a local blockchain using Ganache (Truffle Suite) and deploy a Smart Contract using Metamask and Remix IDE.

### **Tasks to be performed**

1. Install Ganache
2. Connect Ganache accounts with Metamask
3. Connect Remix IDE with Metamask
4. Create a simple Solidity smart contract based on the Mini Project chosen
5. Compile and deploy the smart contract via Ganache accounts added to Metamask
6. Check the transaction details on the Ganache environment
7. Interact with the smart contract

### **THEORY:**

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

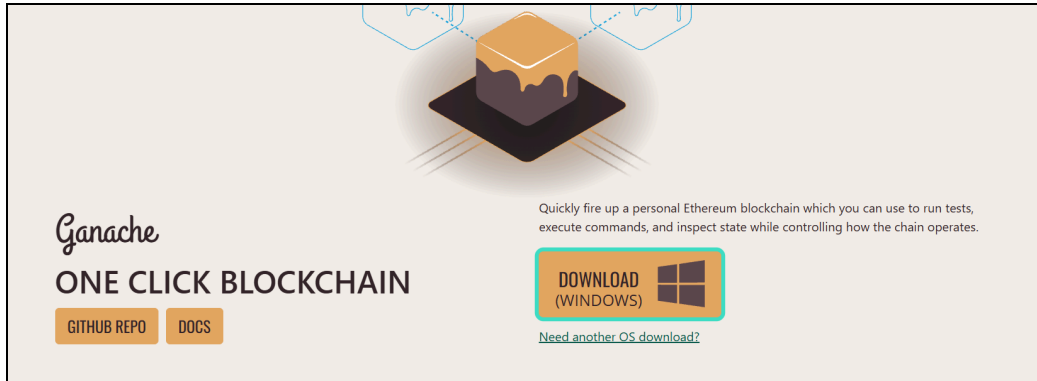
- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

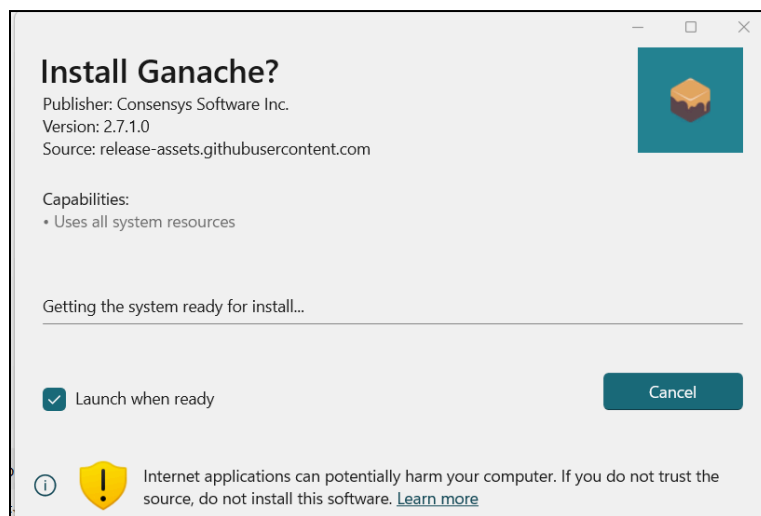
## **Steps to Connect Ganache with Metamask and Remix IDE**

### **1. Install Ganache**

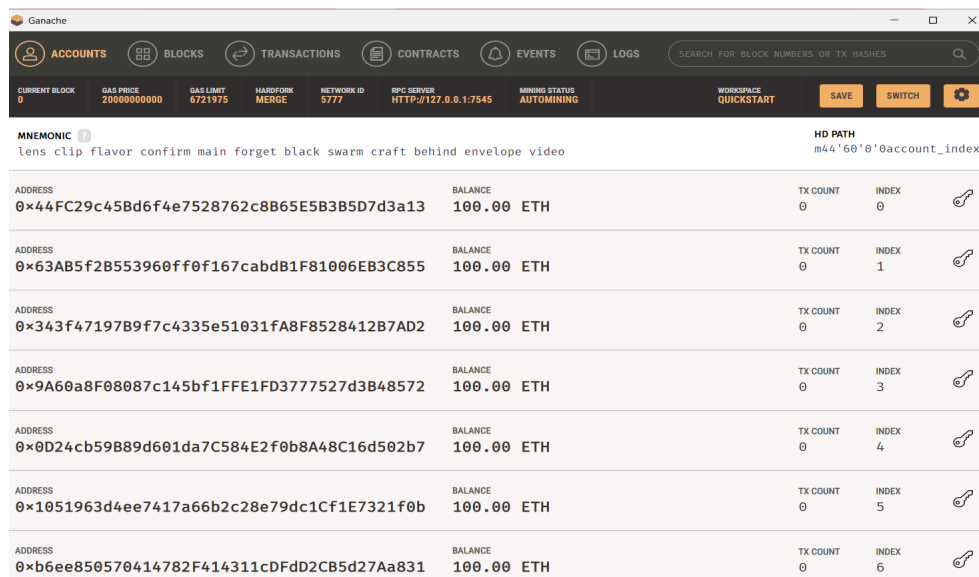
- Download and install Ganache from the Truffle Suite website.
- Launch it to start a local blockchain network with pre-funded test Ether accounts.



**Fig. 1: Ganache download page (Truffle Suite)**



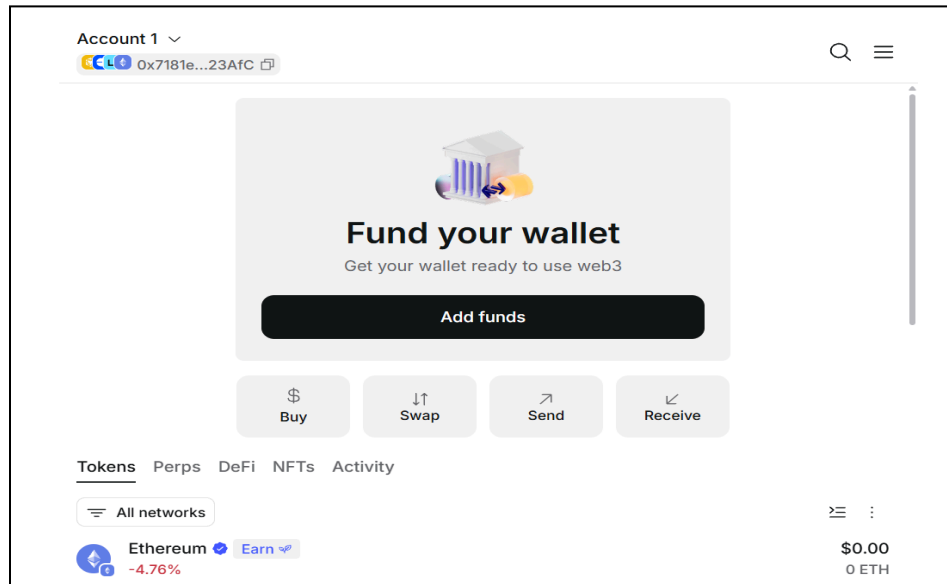
**Fig. 2: Ganache installation prompt (Version 2.7.1.0)**



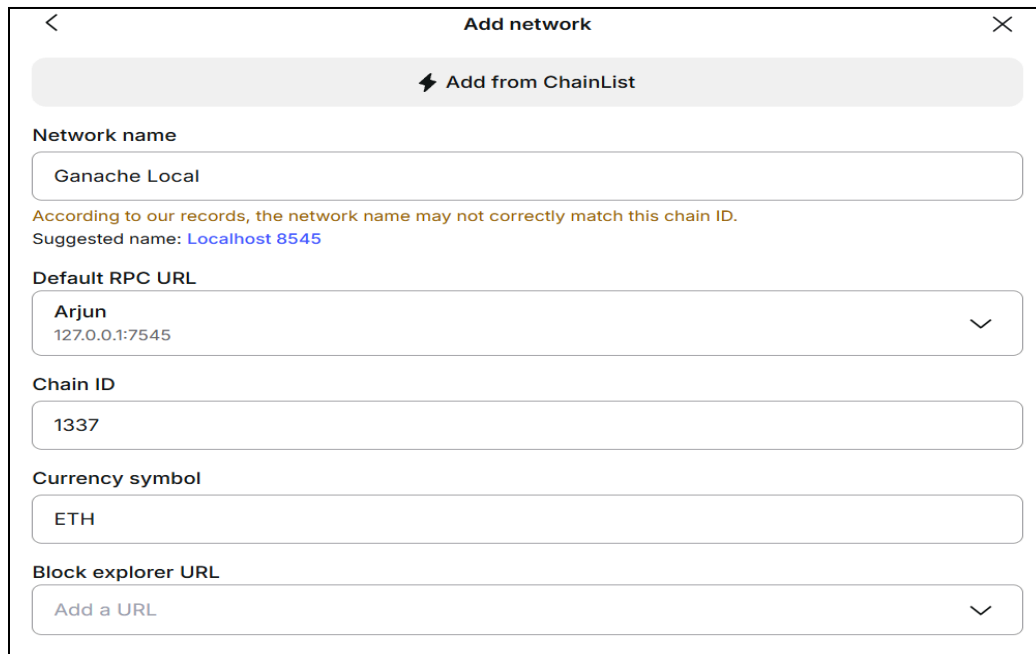
**Fig. 3: Ganache running: 10 test accounts with 100 ETH each, RPC server HTTP://127.0.0.1:7545**

## 2. Connect Ganache Accounts with Metamask

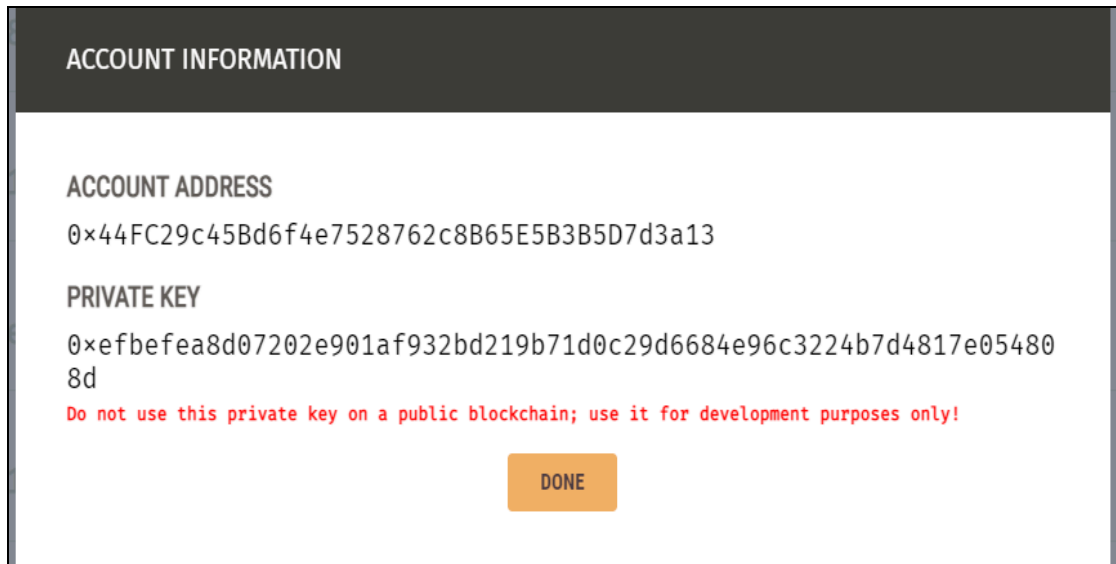
- Open Metamask and add a Custom RPC Network.
- Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
- Import the private key of a Ganache account into Metamask.



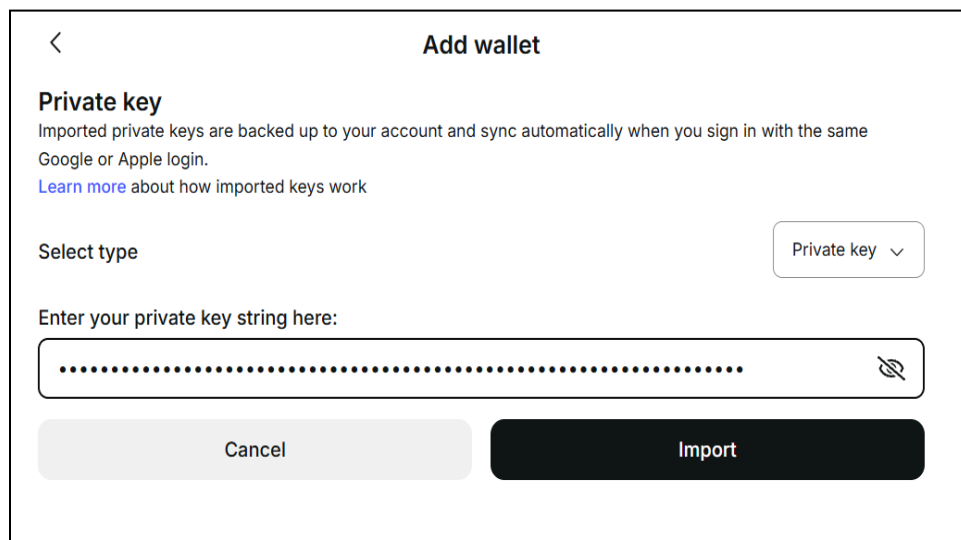
*Fig. 4: Metamask wallet before connecting to Ganache (0 ETH)*



*Fig. 5: Adding the Ganache Local network (RPC URL 127.0.0.1:7545, currency ETH)*



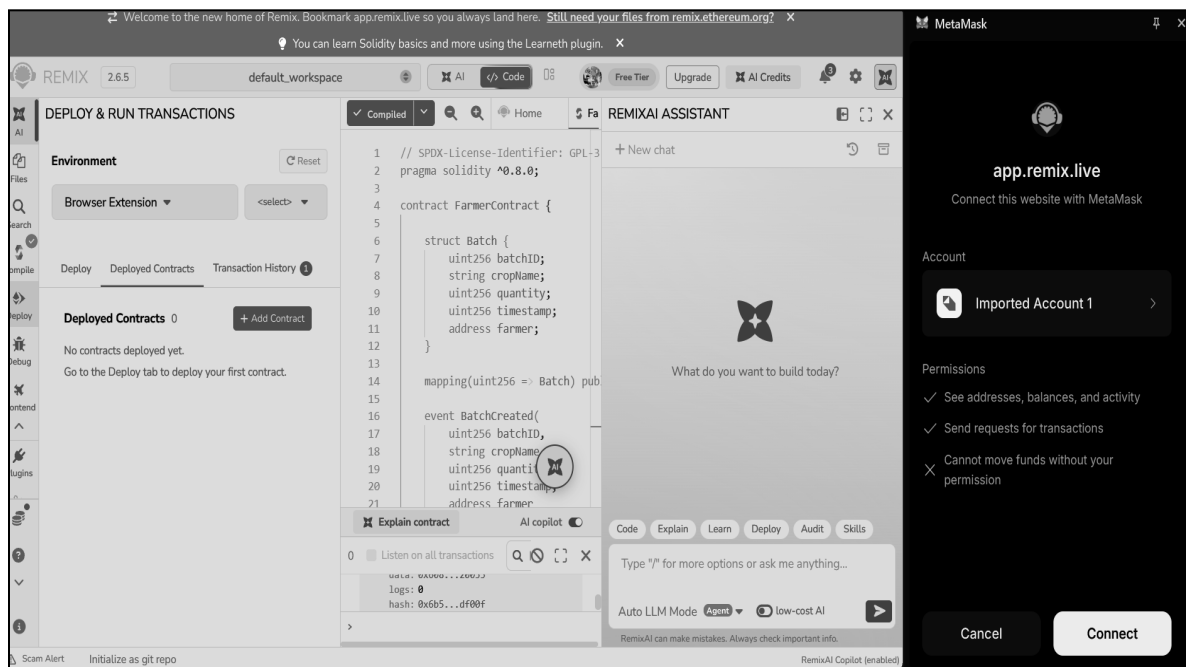
*Fig. 6: Ganache account information showing the account address and private key*



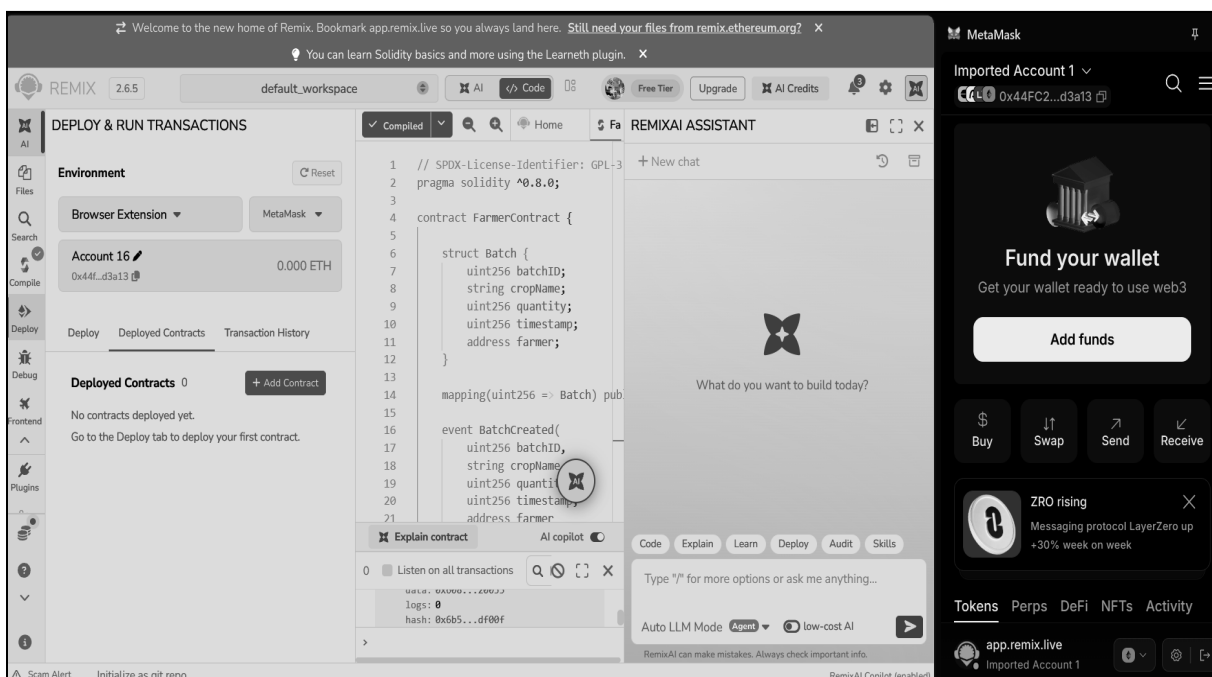
*Fig. 7: Importing the Ganache account into Metamask using its private key*

### 3. Connect Remix IDE with Metamask

- Open Remix IDE in the browser.
- In the “Deploy & Run Transactions” tab, choose **Injected Provider / Browser Extension (Metamask)** as the environment.
- Remix will connect to the Metamask account (linked with Ganache) once the connection is approved.



**Fig. 8: Metamask asking permission to connect with app.remix.live**



**Fig. 9: Remix connected to Metamask: imported Ganache account shown in Deploy & Run Transactions**

#### 4. Create a Simple Solidity Smart Contract

- Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.). A basic example:

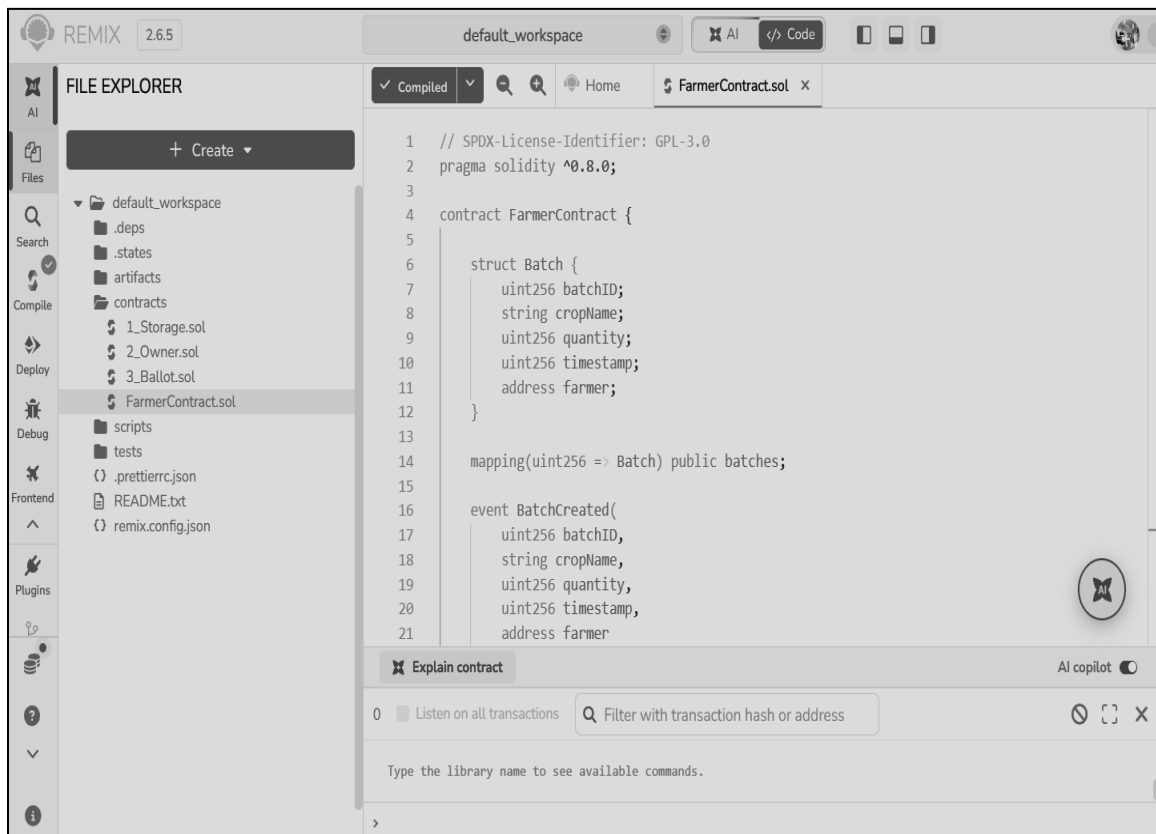
```
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint public data;

    function set(uint x) public {
        data = x;
    }

    function get() public view returns (uint) {
        return data;
    }
}
```

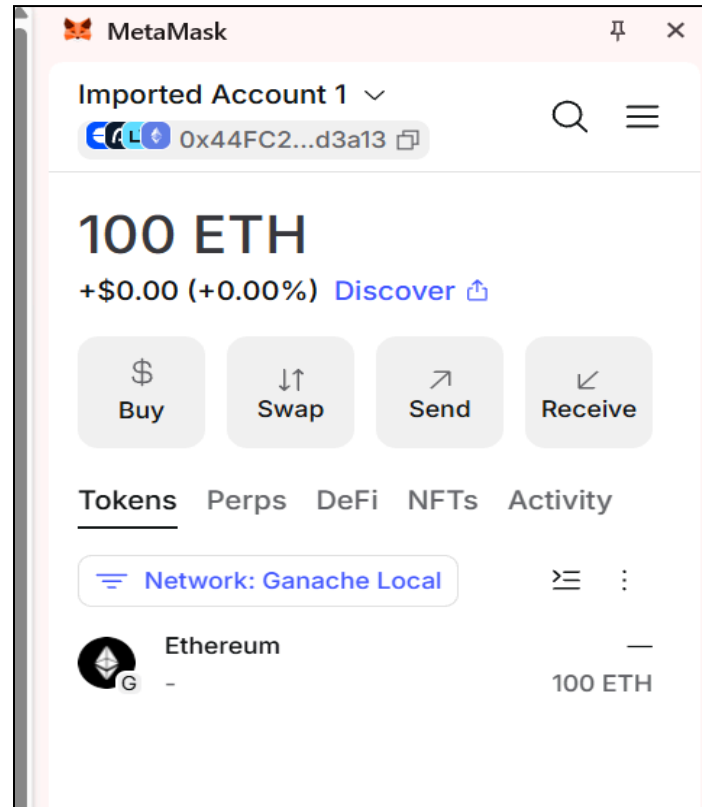
For this mini project (Agricultural Supply Chain), FarmerContract.sol was created in the Remix file explorer under the contracts folder.



**Fig. 10: FarmerContract.sol created in Remix IDE**

## 5. Compile and Deploy the Contract

- Compile the contract using the Solidity compiler in Remix.
- Deploy via the Metamask account connected to Ganache.
- Approve the transaction in Metamask.



*Fig. 11: Metamask on the Ganache Local network showing 100 ETH for the imported account*

## 6. Check Transaction Details on Ganache

- View mined transactions, block details and balances in the Ganache interface.

The mined transaction details for the contract call are shown in Fig. 12 under the Output section.

## 7. Interact with the Smart Contract

- Use Remix to call the contract functions (e.g., addBatch() or getBatch()).
- Observe the state changes and the corresponding transactions.

The addBatch() interaction is shown in Fig. 11 under the Output section.

## Implementation

### FarmerContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract FarmerContract {
    struct Batch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
    }

    mapping(uint256 => Batch) public batches;

    event BatchCreated(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        uint256 timestamp,
        address farmer
    );

    function addBatch(uint256 _batchID, string memory _cropName, uint256
    _quantity) public {
        require(_batchID != 0, "Invalid batch ID");
        require(batches[_batchID].batchID == 0, "Batch already exists!");

        batches[_batchID] = Batch({
            batchID: _batchID,
            cropName: _cropName,
            quantity: _quantity,
            timestamp: block.timestamp,
            farmer: msg.sender
        });

        emit BatchCreated(_batchID, _cropName, _quantity, block.timestamp,
        msg.sender);
    }

    function getBatch(uint256 _batchID) public view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address
    ) {
        Batch memory b = batches[_batchID];
        require(b.batchID != 0, "Batch does not exist!");
        return (b.batchID, b.cropName, b.quantity, b.timestamp, b.farmer);
    }
}
```

**CustomerContract.sol**

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface IRetailerContract {
    function getRetailBatch(uint256 _batchID) external view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address
    );
}

contract CustomerContract {
    struct PurchasedBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
        address retailer;
        address customer;
    }

    mapping(uint256 => PurchasedBatch) public purchasedBatches;
    IRetailerContract public retailerContract;

    event BatchPurchased(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        address retailer,
        address customer,
        uint256 timestamp
    );

    constructor(address _retailerContractAddress) {
        retailerContract = IRetailerContract(_retailerContractAddress);
    }

    function purchaseFromRetailer(uint256 _batchID) public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            // ignore retailer timestamp
            address farmer,
            address distributor,
            address retailer
        ) = retailerContract.getRetailBatch(_batchID);
```

```
        require(id != 0, "Batch does not exist in RetailerContract");
        require(purchasedBatches[_batchID].batchID == 0, "Batch already
        purchased!");

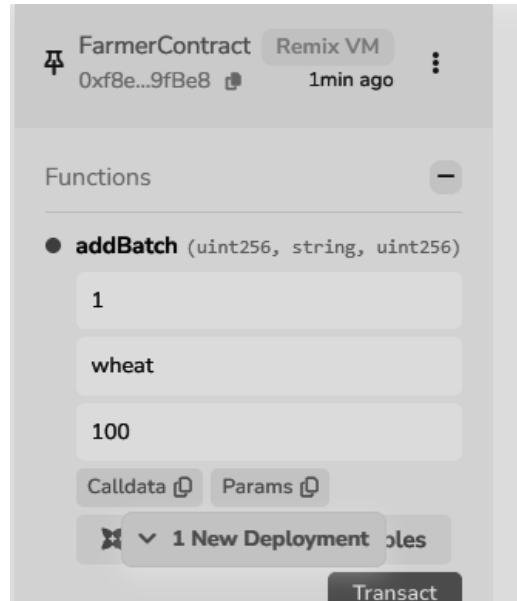
        purchasedBatches[_batchID] = PurchasedBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: distributor,
            retailer: retailer,
            customer: msg.sender
        });

        emit BatchPurchased(
            id, cropName, quantity, farmer, distributor, retailer, msg.sender,
            block.timestamp
        );
    }

    function getPurchasedBatch(uint256 _batchID) public view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address,
        address
    ) {
        PurchasedBatch memory pb = purchasedBatches[_batchID];
        require(pb.batchID != 0, "Batch not purchased!");
        return (
            pb.batchID,
            pb.cropName,
            pb.quantity,
            pb.timestamp,
            pb.farmer,
            pb.distributor,
            pb.retailer,
            pb.customer
        );
    }
}
```

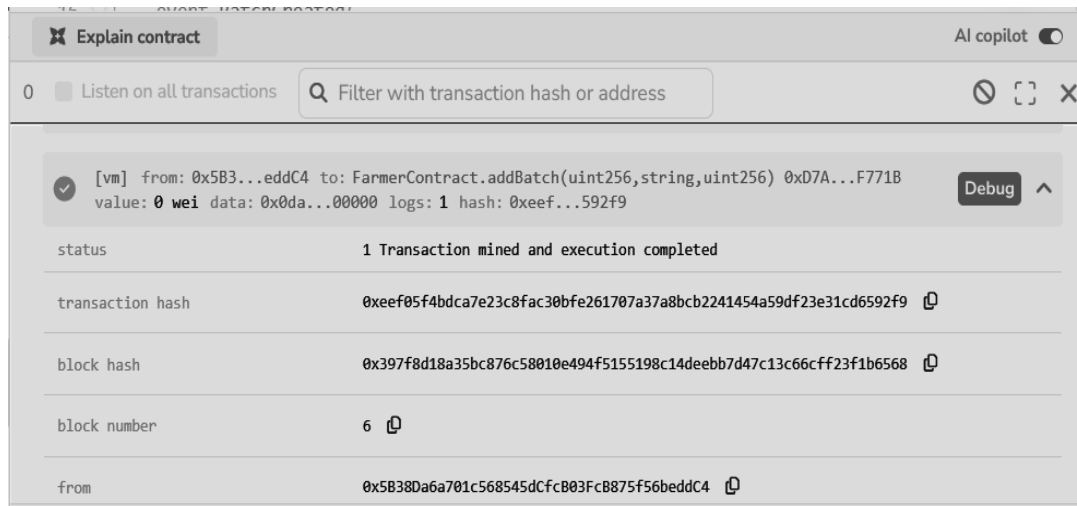
## Output

FarmerContract was deployed and the addBatch() function was called with the inputs batchID = 1, cropName = "wheat" and quantity = 100.



*Fig. 12: FarmerContract deployed; addBatch(1, "wheat", 100) ready to transact*

The transaction was mined successfully (status 1) and the BatchCreated event was logged.



*Fig. 13: Transaction details: transaction mined and execution completed (block number 6)*

## Conclusion

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, developers can deploy, test and debug smart contracts efficiently before moving to a public network.